

SELLER AND INVENTORY MANAGEMENT SYSTEM

Introduction:

An e-commerce platform manages products from different sellers and keeps track of the available stock. The Seller and Inventory Management System is designed to store seller information and monitor the quantity and availability of products.

The system helps to maintain seller details, connect sellers with products, and keep inventory information updated. It also provides useful reports about sellers, products, and stock levels.

Seller Table Creation:

The Seller table stores the basic information of sellers who provide products through the e-commerce platform. It contains seller ID, seller name, email, phone number, and address

Inventory Table Creation:

The Inventory table is used to maintain the stock details of products. It stores the inventory ID, product ID, seller ID, stock quantity, stock status, and last updated date.

```
SQL> CREATE TABLE Seller(seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone NUMBER(10), Address VARCHAR(45));
Table created.

SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7), Product_ID VARCHAR(7), Seller_ID VARCHAR(7), Stock_Quantity number, Stock_status VARCHAR(11), Last_Updated DATE);
Table created.
```

Relationship Between Tables:

The Seller and Inventory tables are connected with the Product table to maintain proper data relationships.

Seller – Product Relationship

A seller can provide multiple products. Therefore, the relationship between Seller and Product can be represented as one-to-many.

Product – Inventory Relationship

Inventory stores the stock information related to each product. The Product and Inventory relationship is represented as one-to-one in this system.

Defining Primary Keys:

Primary keys are added to uniquely identify each record in the Seller and Inventory tables.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);  
Table altered.
```

Defining Foreign Keys:

Foreign keys are used to establish connections between the Inventory table and the existing Product and Seller tables.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

Unique Email Constraint:

Each seller should have a unique email address.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

NOT NULL Constraint:

The seller email field is made mandatory.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

Stock Quantity Validation:

The stock quantity should not contain a negative value.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK (Stock_Quantity >=0);  
Table altered.
```

Adding Seller Records:

Seller details are inserted into the Seller table using the following query:

```
SQL> INSERT INTO SELLER VALUES ('S301','Arun Kumar','arun@hotel.com','9000000001','Chennai');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S302','Priya Devi','priya@hotel.com','9000000002','Chennai');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S303','Karthik Raj','karthik@hotel.com','9000000003','Madurai');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S304','Meena Sri','meena@hotel.com','9000000004','Salem');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S305','Ravi Kumar','ravi@hotel.com','9000000005','Coimbatore');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S306','Divya Raj','divya@hotel.com','9000000006','Trichy');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S307','Suresh Babu','suresh@hotel.com','9000000007','Chennai');
1 row created.

SQL> INSERT INTO SELLER VALUES ('S308','Anitha Devi','anitha@hotel.com','9000000008','Vellore');
1 row created.

SQL> SELECT * FROM Seller;
```

SELLER_	SELLER_NAME	EMAIL	PHONE	ADDRESS
S301	Arun Kumar	arun@hotel.com	9000000001	Chennai
S302	Priya Devi	priya@hotel.com	9000000002	Chennai
S303	Karthik Raj	karthik@hotel.com	9000000003	Madurai
S304	Meena Sri	meena@hotel.com	9000000004	Salem
S305	Ravi Kumar	ravi@hotel.com	9000000005	Coimbatore
S306	Divya Raj	divya@hotel.com	9000000006	Trichy
S307	Suresh Babu	suresh@hotel.com	9000000007	Chennai
S308	Anitha Devi	anitha@hotel.com	9000000008	Vellore
S309	Manoj Kumar	manoj@hotel.com	9000000009	Erode
S310	Naveen Raj	naveen@hotel.com	9000000010	Tirunelveli

```
10 rows selected.
```

Adding Inventory Records:

Inventory records are created using the products already available in the Product table.

```

SQL> INSERT INTO INVENTORY VALUES ('I301','P301','S301',1,'Available',TO_DATE('20-09-2026','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO INVENTORY VALUES ('I302','P302','S302',1,'Available',TO_DATE('21-09-2026','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO INVENTORY VALUES ('I303','P303','S303',1,'Available',TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO INVENTORY VALUES ('I304','P304','S304',0,'Booked',TO_DATE('23-09-2026','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO INVENTORY VALUES ('I305','P305','S305',1,'Available',TO_DATE('24-09-2026','DD-MM-YYYY'));
1 row created.

SQL>
SQL> INSERT INTO INVENTORY VALUES ('I306','P306','S306',0,'Booked',TO_DATE('24-09-2026','DD-MM-YYYY'));
1 row created.

```

```
SQL> SELECT * FROM Inventory;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATU	LAST_UPDA
-----	-----	-----	-----	-----	-----
I301	P301	S301	1	Available	20-SEP-26
I302	P302	S302	1	Available	21-SEP-26
I303	P303	S303	1	Available	22-SEP-26
I304	P304	S304	0	Booked	23-SEP-26
I305	P305	S305	1	Available	24-SEP-26
I306	P306	S306	0	Booked	24-SEP-26
I308	P308	S308	1	Available	25-SEP-26
I309	P309	S309	0	Booked	26-SEP-26
I310	P310	S310	1	Available	26-SEP-26

```
9 rows selected.
```

Seller wise product:

This report provides information about the number of products handled by each seller and the total stock available with them.

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Ravi Kumar	1	1
Priya Devi	1	1
Anitha Devi	1	1
Naveen Raj	1	1
Arun Kumar	1	1
Karthik Raj	1	1

6 rows selected.

Stock Availability Report:

This report displays the product name along with its current stock quantity and availability status.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATU
Room 101	1	Available
Room 102	1	Available
Room 201	1	Available
Room 202	0	Booked
Room 301	1	Available
Room 302	0	Booked
Room 402	1	Available
Room 501	0	Booked
Room 601	1	Available

9 rows selected.

Inventory Analysis:

Inventory analysis is performed to understand the overall stock position of the products.

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
6

Products Currently Out of Stock:

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';
```

OUTOFSTOCK_PRODUCTS
0

Product with Highest Stock:

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);
```

PRODUCT_NAME	STOCK_QUANTITY
Room 101	1
Room 102	1
Room 201	1
Room 301	1
Room 402	1
Room 601	1

6 rows selected.

Average Stock Quantity:

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;

AVERAGE_INVENTORY
-----
.666666667
```

Conclusion:

The Seller and Inventory Management System provides an organized method for maintaining seller and stock information. The database uses primary keys, foreign keys, and constraints to maintain data consistency. The inventory reports help identify available products, stock quantities, out-of-stock products, highest-stocked products, and average inventory levels.